

Master Coroutines

The Simple Way - Kotlin Coroutines explained in small parts, with everyday analogies

Mohammed Fahim Khan

[linkedin.com/in/ifahimkhan](https://www.linkedin.com/in/ifahimkhan) | github.com/ifahimkhan

No prior knowledge of threads, RxJava, or concurrency needed. Read the parts in order. Each part is short. Each part builds on the one before it.

Contents

1. Part 1: What are Coroutines?
2. Part 2: Why do we need Coroutines?
3. Part 3: Suspend Function
4. Part 4: Dispatchers
5. Part 5: Launch vs Async
6. Part 6: withContext
7. Part 7: Scopes
8. Part 8: CoroutineContext
9. Part 9: Exception Handling
10. Part 10: Flow - The Basics

How to read this

- Read one part a day. Do not rush.
- Type the code yourself. Do not copy-paste.
- If a part feels heavy, go back one part. The gap is usually there.

Part 1: What are Coroutines?

Let's get started.

Break the word first

Coroutines = Co + Routines

Here, **Co** means **cooperation** and **Routines** means **functions**.

So when functions cooperate with each other, we call it Coroutines.

That's the whole idea. Everything else is detail.

The restaurant analogy

Imagine a restaurant with one waiter.

A customer orders food. The waiter gives the order to the kitchen.

Now, what should the waiter do?

Option 1: Stand near the kitchen and wait for the food.

The waiter is doing nothing for 10 minutes. Other customers wait. They get angry. This is a **blocked thread**.

Option 2: Give the order to the kitchen, then go take orders from other tables. When the food is ready, come back and serve it.

Same one waiter. Same one kitchen. But nobody is standing idle.

Option 2 is a coroutine.

The waiter did not stop. The waiter **paused** one job and picked up another job, then came back later.

Cooperation, in code

Let's see the same idea with two functions, `functionA` and `functionB`.

```
fun functionA() {
    taskA1()
    // now let functionB do some work
    functionB()
    // and come back here to continue
    taskA2()
}

fun functionB() {
    taskB1()
    taskB2()
}
```

Here, `functionA` does `taskA1`, then hands over control to `functionB`.

`functionB` does its work and gives control back to `functionA`.

They are **cooperating**.

With Kotlin Coroutines, this handing over of control happens automatically. We do not write it by hand.

Coroutines vs Threads

Both do many things at once. So what is different?

Think of it like this:

- A **thread** is a full worker hired by the operating system. Hiring one is expensive. Keeping 1000 of them is very expensive.
- A **coroutine** is a **task on a sticky note**. A worker picks up a note, does some of it, sticks it back, picks another note.

We can create thousands of sticky notes. They cost almost nothing.

This is why we say:

Coroutines are lightweight threads.

Lightweight means a coroutine does not map to a real OS thread of its own. So the processor does not have to do context switching for it. That makes it fast and cheap.

Stackless vs Stackful

Coroutines exist in many languages. There are two types:

- **Stackless** - the coroutine does not carry its own stack.
- **Stackful** - the coroutine carries its own stack.

Kotlin implements **stackless** coroutines. That is exactly why they don't map onto a native thread.

You do not need to remember this on day one. Just know the word if someone asks in an interview.

What the official Kotlin docs say

Now this paragraph should make sense to you:

One can think of a coroutine as a light-weight thread. Like threads, coroutines can run in parallel, wait for each other and communicate. The biggest difference is that coroutines are very cheap, almost free: we can create thousands of them, and pay very little in terms of performance. True threads, on the other hand, are expensive to start and keep around.

One important line

Coroutines do not replace threads.

Threads are still there underneath. Coroutines are a **framework to manage them** in a smarter way.

The exact definition

A framework to manage concurrency in a more performant and simple way with its lightweight thread, which is written on top of the actual threading framework, to get the most out of it by taking advantage of the cooperative nature of functions.

Read that once now. Read it again after Part 5. It will feel much lighter then.

Thumb-rules

- Coroutines = functions cooperating with each other.
- A coroutine can pause and resume. A blocked thread cannot.
- Coroutines are cheap. Threads are expensive.
- Coroutines run on top of threads, they do not remove them.

Next: [Why do we need Coroutines?](#)

That's it for now.

Part 2: Why do we need Coroutines?

In Part 1, we learned what coroutines are.

Now let's see the actual problem they solve. Once you feel the problem, the solution sticks.

The very standard use case

Almost every app does this:

1. Fetch the user from the server.
2. Show the user on the screen.

Simple. Let's write it.

```
fun fetchAndShowUser() {  
    val user = fetchUser()  
    showUser(user)  
}  
  
fun fetchUser(): User {  
    // make network call  
    // return user  
}  
  
fun showUser(user: User) {  
    // show user  
}
```

Looks clean. But run it in Android and it crashes with:

```
NetworkOnMainThreadException
```

Why does it crash?

Android has one special thread called the **main thread** (also called the UI thread).

Everything you see - buttons, scrolling, animation - is drawn by this one thread.

The analogy: the main thread is the cashier at a shop counter.

A network call takes 2 seconds. If the cashier goes to the warehouse himself to fetch a box, the counter is empty for 2 seconds. Nobody can pay. The queue freezes.

That frozen queue is your frozen app.

So Android simply refuses. It says: *do not do network work on the main thread*.

Fine. So we move the work to a background thread. But now a new problem appears.

Solution 1: Callbacks

We do the work in the background and get the result through a callback.

```
fun fetchAndShowUser() {
    fetchUser { user ->
        showUser(user)
    }
}

fun fetchUser(callback: (User) -> Unit) {
    // make network call on background thread to get user
    callback(user)
}
```

This works. The app does not freeze.

But look what happens when one call depends on another.

```
fun fetchData() {
    fetchA { a ->
        fetchB(a) { b ->
            fetchC(b) { c ->
                // do something with c
            }
        }
    }
}
```

See the shape? It keeps sliding to the right.

This is called **callback hell**.

The analogy: you ask a friend to call you back. When he calls, you ask him to call another friend to call you back. When that friend calls, you ask him to call a third friend. Now try adding error handling to that chain. It becomes a mess.

Now imagine adding a `try-catch` to each level. Now imagine cancelling all of it when the user leaves the screen. That is the real pain.

Solution 2: RxJava

RxJava removes the nesting.

```
fetchUser()
    .subscribeOn(Schedulers.io())
    .observeOn(AndroidSchedulers.mainThread())
    .subscribe { user ->
        showUser(user)
    }
```

Better. No nesting.

But it comes with a price: a big new vocabulary. Operators, Observables, Singles, Schedulers, disposables. Powerful, but a lot to learn just to make one network call.

Solution 3: Coroutines

Now the same thing with coroutines.

```
fun fetchAndShowUser() {
    GlobalScope.launch(Dispatchers.Main) {
        val user = fetchUser() // fetch on IO thread
        showUser(user)         // back on UI thread
    }
}

suspend fun fetchUser(): User {
    return withContext(Dispatchers.IO) {
        // make network call on IO thread
        // return user
    }
}

fun showUser(user: User) {
    // show user
}
```

Look at these two lines:

```
val user = fetchUser()
showUser(user)
```

Top to bottom. One line after another. Exactly like our very first broken version.

But it does not freeze the app.

The code **looks synchronous**, but it **is asynchronous**.

That single sentence is the reason coroutines won.

Note: I have used `GlobalScope` here for a quick example. In a real Android project, we should avoid `GlobalScope` at all costs. We should use scopes like `lifecycleScope` and `viewModelScope`. We will learn why in Part 7.

Compare the three

Approach	Nesting	Learning curve	Reads like normal code
Callbacks	Grows to the right (callback hell)	Small	No
RxJava	Flat	Large	Somewhat
Coroutines	Flat	Small	Yes

Thumb-rules

- The main thread draws the UI. Never make it wait.
- Callbacks fix freezing but create callback hell.
- RxJava fixes nesting but brings a big vocabulary.
- Coroutines fix both - async code that reads top to bottom.

Next: [Suspend Function](#)

That's it for now.

Part 3: Suspend Function

In Part 2, we saw code that looks synchronous but runs asynchronously.

The keyword that makes that magic possible is `suspend`.

What is a suspend function?

A suspend function is a function that can be **started**, **paused**, and **resumed**.

That's it. Three words: start, pause, resume.

The bookmark analogy

You are reading a book. Your phone rings.

You do **not** burn the book. You do **not** sit staring at the page.

You put a **bookmark**, close the book, take the call, then open the book at the exact same line and continue.

A suspend function does the same thing:

- Bookmark = where it paused
- Take the call = the thread goes and does other work
- Continue = it resumes from the exact same line

The key point: while the book is closed, **you are free**. You are not stuck.

That is the difference between **suspending** and **blocking**.

Suspend vs Block - the important difference

This is the one thing beginners mix up. Let's make it clear.

	Blocking	Suspending
What happens to the thread	Thread is stuck, doing nothing	Thread is free, goes and does other work
Example	<code>Thread.sleep(2000)</code>	<code>delay(2000)</code>
Cost	Wastes a whole worker	Wastes nothing

Analogy: you order tea at a shop.

- **Blocking:** you stand at the counter frozen for 5 minutes until the tea is ready. The shopkeeper cannot serve anyone else, because you are in front of him.
- **Suspending:** you take a token and sit down. The shopkeeper serves 4 other people. When your tea is ready, your number is called and you continue.

Same waiting time. Very different use of the shopkeeper.

How to write one

Just add the `suspend` keyword in front of `fun`.

```
suspend fun fetchUser(): User {  
    return withContext(Dispatchers.IO) {  
        // make network call on IO thread  
        // return user  
    }  
}
```

The one rule of suspend functions

A suspend function can only be called from **another suspend function**, or from **inside a coroutine**.

Why? Because pausing and resuming needs someone who knows how to pause and resume. A normal function has no idea how to do that.

Analogy: only a person holding the bookmark can reopen the book. A random passerby cannot.

So how do we call it from `onCreate` ?

`onCreate` is a normal Android function. We cannot add `suspend` to it.

So we start a coroutine, and call the suspend function inside it.

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    GlobalScope.launch(Dispatchers.Main) {  
        val user = fetchUser() // fetch on IO thread  
        showUser(user)         // back on UI thread  
    }  
}
```

`launch { }` opens the door. Inside that door, suspend functions are allowed.

Note: again, `GlobalScope` is only for quick examples. Use `lifecycleScope` in an Activity. Part 7 explains why.

Let's trace the control

This example shows exactly how control moves.

```

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        GlobalScope.launch(Dispatchers.Main) {
            doSomething()          // non-suspend, UI thread
            doLongRunningTask()    // suspend, background thread
            doSomethingElse()      // non-suspend, UI thread
        }
    }

    fun doSomething() {
    }

    fun doSomethingElse() {
    }

    suspend fun doLongRunningTask() {
        withContext(Dispatchers.Default) {
            // long running work
            delay(2000) // added delay to simulate
        }
    }
}

```

Step by step:

1. We launched with `Dispatchers.Main`, so we start on the **UI thread**.
2. `doSomething()` is a normal function - it runs on the **UI thread**.
3. `doLongRunningTask()` is a suspend function using `Dispatchers.Default` - the work moves to a **background thread**. The UI thread is now **free**, so the screen keeps scrolling smoothly.
4. When that work finishes, control comes **back to the UI thread**.
5. `doSomethingElse()` runs on the **UI thread**.

Three lines, written top to bottom. Two thread switches happened. We wrote zero callbacks.

This is the beauty of Kotlin Coroutines.

A common beginner mistake

Adding `suspend` does **not** move work to a background thread by itself.

```

suspend fun wrong(): User {
    // still runs on whatever thread called it
    return heavyWork()
}

```

`suspend` only means "this function can pause".

Which thread it runs on is decided by the Dispatcher. That is our next part.

Thumb-rules

- `suspend` = can start, pause, and resume.
- Suspending frees the thread. Blocking wastes it.
- Use `delay()` , not `Thread.sleep()` , inside coroutines.
- A suspend function is callable only from a coroutine or another suspend function.
- `suspend` alone does not change the thread.

Next: [Dispatchers](#)

That's it for now.

Part 4: Dispatchers

At the end of Part 3 we said: `suspend` does not decide the thread.

So who decides? **Dispatchers**.

Dispatchers help Coroutines in deciding the thread on which the task has to be done.

The office analogy

Think of an office with three rooms:

- **The front desk** - faces the customer. Must always be free and smiling. Never give it heavy work.
- **The back office** - people here make phone calls, send emails, wait for replies. Lots of waiting, little thinking.
- **The lab** - people here do heavy calculation. No waiting, pure brain work.

Now, a Dispatcher is simply the person who says: *"This job goes to that room."*

Kotlin gives us these rooms ready-made.

Dispatchers.Main - the front desk

The Android **UI thread**.

Use it for:

- Updating the UI
- Small, quick, in-memory work

```
launch(Dispatchers.Main) {  
    // update the UI  
}
```

Rule: get in, do the small thing, get out.

Dispatchers.IO - the back office

IO means Input / Output. Work where we mostly **wait** for someone else.

Use it for:

- Network calls (downloading a file, hitting an API)
- Database read and write
- Reading and writing files
- Reading SharedPreferences

```
launch(Dispatchers.IO) {  
    // network call or database work  
}
```

Why is this a separate room? Because waiting is cheap. We can have many people waiting at once, so this pool is allowed to be **big**.

Dispatchers.Default - the lab

CPU-intensive work. Data is already in memory, we just need to think hard about it.

Use it for:

- Sorting a large list
- Matrix calculation
- Parsing or transforming a big JSON already in memory
- Bitmap / image processing

```
launch(Dispatchers.Default) {  
    // heavy calculation  
}
```

This pool is **small** on purpose - roughly the number of CPU cores you have. More thinkers than brains does not help.

Dispatchers.Unconfined - the odd one

It does not stick to any particular thread. It just continues on whichever thread is available at the moment.

Mostly useful in **unit testing**.

```
launch(Dispatchers.Unconfined) {  
    // not confined to any specific thread  
}
```

As a beginner: know it exists, don't reach for it.

The full picture

Dispatcher	Room	Use it for	Pool size
<code>Dispatchers.Main</code>	Front desk	UI updates, tiny work	1 thread (the UI thread)
<code>Dispatchers.IO</code>	Back office	Network, database, files	Large
<code>Dispatchers.Default</code>	Lab	Heavy CPU work in memory	~number of CPU cores
<code>Dispatchers.Unconfined</code>	No fixed room	Testing	-

If you know RxJava

Dispatchers in Kotlin Coroutines are like Schedulers in RxJava:

Coroutines	RxJava
<code>Dispatchers.Default</code>	<code>Schedulers.computation()</code>
<code>Dispatchers.IO</code>	<code>Schedulers.io()</code>
<code>Dispatchers.Main</code>	<code>AndroidSchedulers.mainThread()</code>

If you don't know RxJava, skip this table. You lose nothing.

The pattern to remember

Every one of them follows the same shape:

```
launch(Dispatchers.WhichRoom) {  
    // task that belongs in that room  
}
```

The mistake everyone makes once

Using `Dispatchers.IO` for heavy calculation, or `Dispatchers.Default` for network calls.

It will still work. Nothing crashes. But it wastes resources.

Easy way to decide - ask one question:

Is my code waiting, or is my code thinking?

- **Waiting** (for server, disk, database) → `Dispatchers.IO`
- **Thinking** (sorting, math, image work) → `Dispatchers.Default`
- **Showing** (UI) → `Dispatchers.Main`

Thumb-rules

- Dispatchers decide **which thread** the work runs on.
- `Main` = UI. `IO` = waiting. `Default` = thinking. `Unconfined` = testing.
- Never put heavy work on `Main`.
- Waiting or thinking - that one question picks your dispatcher.

Next: [Launch vs Async](#)

That's it for now.

Part 5: Launch vs Async

Both `launch` and `async` are functions in Kotlin to **start** a coroutine.

```
launch { }  
async { }
```

So what is different?

The one-line answer

- `launch` : **fire and forget**
- `async` : **perform a task and return a result**

The post office analogy

- `launch` is like **posting a letter**. You drop it in the box and walk away. You get a receipt slip, but no reply comes back to you.
- `async` is like **ordering food with a token number**. You get a token. Later you show the token at the counter and collect your food.

That token is `Deferred`. Showing the token is `await()`.

launch - fire and forget

```
val job = GlobalScope.launch(Dispatchers.Default) {  
    // do something and do not return result  
}
```

`launch` returns a `Job`.

A `Job` is the receipt slip. With it you can:

- check the status of the task
- cancel the task

```
job.cancel()
```

But a `Job` carries **no result value**.

Use `launch` when the answer is "just go do it": save to database, log an event, update the UI.

async - give me the result

```
val deferredJob = GlobalScope.async(Dispatchers.Default) {
    // do something and return result, for example 10 as a result
    return@async 10
}

val result = deferredJob.await() // result = 10
```

`async` returns a `Deferred<T>`.

`Deferred` is the token. Call `await()` on it to collect the value.

If you know Java, this is like `Future` - where you call `future.get()` to get the result.

And `Deferred` is also a `Job`, so you can still check status or cancel it.

Note: I have used `GlobalScope` for quick examples. We should avoid using it at all costs. In an Android project, we should use custom scopes based on our use case, such as `lifecycleScope` and `viewModelScope`. Part 7 covers this.

Doing two tasks in parallel

This is where `async` really shines.

Say we have two tasks, each taking 2 seconds.

One after another (4 seconds):

```
launch {
    val one = doTaskOne()    // 2 seconds
    val two = doTaskTwo()    // 2 more seconds
    showResult(one + two)    // total ~4000 ms
}
```

Both at the same time (2 seconds):

```
launch {
    val deferredOne = async { doTaskOne() }
    val deferredTwo = async { doTaskTwo() }
    val result = deferredOne.await() + deferredTwo.await()
    showResult(result)      // total ~2000 ms
}
```

Analogy: you need tea and a sandwich.

- Sequential: order tea, wait, drink it, then order the sandwich, wait.
- Parallel: order both, then collect both.

Same shop, half the time.

Important: start both `async` blocks **first**, and only then call `await()`. If you write `async { }.await()` on one line, you have gone back to sequential without noticing.

The second difference - exceptions

This one surprises people.

Say we have a function that throws:

```
private fun doSomethingAndThrowException() {  
    throw Exception("Some Exception")  
}
```

With `launch` :

```
GlobalScope.launch {  
    doSomethingAndThrowException()  
}
```

It will **CRASH** the application, as expected.

With `async` :

```
GlobalScope.async {  
    doSomethingAndThrowException()  
}
```

The application will **NOT** crash. The exception is stored inside the `Deferred` and gets **silently dropped** unless we handle it.

Analogy: `launch` is a smoke alarm - it screams. `async` is a note left inside a sealed envelope - if nobody opens the envelope, nobody ever knows.

You open the envelope by calling `await()`.

Both can be handled with `try-catch` :

```
GlobalScope.launch {  
    try {  
        doSomethingAndThrowException()  
    } catch (e: Exception) {  
        // handle exception  
    }  
}
```

```
GlobalScope.async {  
    try {  
        doSomethingAndThrowException()  
    } catch (e: Exception) {  
        // handle exception  
    }  
}
```

Part 9 goes deeper into this.

The table

	launch	async
Purpose	Fire and forget	Perform a task and return a result
Returns	<code>Job</code> - no result value	<code>Deferred<T></code> - <code>await()</code> gives the result
Exception	Crashes the app if not handled	Stored in the <code>Deferred</code> , silently dropped unless handled

Thumb-rules

- Need a result? Use `async` + `await()` .
- Don't need a result? Use `launch` .
- Both can run tasks in parallel.
- Start all `async` blocks first, `await()` afterwards.
- An unhandled exception in `launch` is loud. In `async` it is silent.

Next: [withContext](#)

That's it for now.

Part 6: withContext

We have seen `launch` and `async`. Both **create a new coroutine**.

Now meet the third one, which does something different.

`withContext` is a suspend function through which we can do a task by providing the Dispatcher on which we want the task to be done.

The key sentence:

`withContext` does **not** create a new coroutine. It only **shifts the context** of the existing coroutine.

The analogy

You are already at work. A job needs to be done in the lab.

- `launch` / `async` = **hire a new person** to go do it.
- `withContext` = **you yourself walk into the lab**, do the work, and walk back to your desk.

Same person. Different room. Nobody new was hired.

The basic form

```
private suspend fun doLongRunningTaskAndDoNotReturnResult() {
    withContext(Dispatchers.Default) {
        // your code for doing a long running task
        delay(2000) // added delay to simulate
    }
}
```

It can also return a result:

```
private suspend fun doLongRunningTask(): Int {
    return withContext(Dispatchers.Default) {
        // your code for doing a long running task
        delay(2000) // added delay to simulate
        return@withContext 10
    }
}
```

Since `withContext` is a suspend function, it can only be called from a suspend function or a coroutine. That is why both functions above are marked `suspend`.

Using it:

```
GlobalScope.launch(Dispatchers.Main) {  
    val result = doLongRunningTask()  
    showResult(result) // back on UI thread  
}
```

Notice: after `withContext` finishes, we are automatically **back on the UI thread**. We did not write a single line to come back.

withContext is sequential

Two tasks, 2 seconds each:

```
GlobalScope.launch(Dispatchers.Main) {  
    val resultOne = doLongRunningTaskOne()  
    val resultTwo = doLongRunningTaskTwo()  
    showResult(resultOne + resultTwo)  
}
```

This takes **about 4000 milliseconds**. Task two starts only after task one is done.

That is expected - one person cannot be in two rooms at once.

For parallel work, we need two coroutines, so we go back to `async` :

```
GlobalScope.launch {  
    val deferredOne = async { doLongRunningTaskOne() }  
    val deferredTwo = async { doLongRunningTaskTwo() }  
    val result = deferredOne.await() + deferredTwo.await()  
    showResult(result)  
}
```

Now it takes **about 2000 milliseconds**.

The best pattern in Android

This is the pattern you will use every day. Learn this shape:

```
// The caller does not care about threads at all
lifecycleScope.launch(Dispatchers.Main) {
    val user = fetchUser()
    showUser(user)
}

// The function itself decides where its work belongs
suspend fun fetchUser(): User {
    return withContext(Dispatchers.IO) {
        // network call
    }
}
```

The rule behind it: **a suspend function should be safe to call from anywhere.**

The function knows it needs IO. So the function puts `withContext(Dispatchers.IO)` inside itself. The caller just calls it and stays clean.

This is called being **main-safe**.

The three, side by side

	Creates a new coroutine?	Returns	Runs in parallel?	Is it a suspend function?
<code>launch</code>	Yes	<code>Job</code>	Yes	No
<code>async</code>	Yes	<code>Deferred<T></code>	Yes	No
<code>withContext</code>	No	The result directly	No (sequential)	Yes

Thumb-rules

- `launch` and `async` create coroutines. `withContext` does not.
- `withContext` just moves the current coroutine to another Dispatcher and brings it back.
- `withContext` is sequential. For parallel work, use `async`.
- Put `withContext` **inside** the suspend function, so callers never worry about threads.

Next: [Scopes](#)

That's it for now.

Part 7: Scopes

In every part so far, I kept writing the same note:

Note: avoid `GlobalScope`. Use `lifecycleScope` or `viewModelScope`.

Now let's finally understand why.

The problem

A user opens a screen. A network call starts. It takes 3 seconds.

After 1 second, the user presses back. The screen is destroyed.

But the network call is **still running**.

When it finishes, it tries to update a screen that no longer exists.

Result: wasted battery, wasted data, leaked memory, and sometimes a crash.

The analogy

A **scope** is a **contract with an end date**.

Think of hiring workers for a construction site.

- **GlobalScope** = the worker is hired **forever**. Even after the building is demolished, he keeps showing up and hammering the air. Nobody ever told him to stop.
- **A proper scope** = the worker is hired **for this building only**. Building gone → contract over → worker goes home automatically.

That automatic "go home" is the whole point of scopes.

What is a scope, technically?

A scope defines the **lifetime** of the coroutines started inside it.

When the scope is cancelled, **every coroutine inside it is cancelled too**.

You do not have to track them one by one. That is the gift.

Activity - use `lifecycleScope`

The Activity is the scope. When the Activity is destroyed, the work should stop.


```

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        lifecycleScope.launch {
            val user = fetchUser()
            // show user
        }
    }

    suspend fun fetchUser(): User {
        return withContext(Dispatchers.IO) {
            // fetch user
            // return user
        }
    }
}

```

As soon as the Activity is destroyed, the task gets cancelled if it is still running - because we used a scope bound to the **lifecycle of the Activity**.

ViewModel - use viewModelScope

The ViewModel is the scope. When the ViewModel is cleared, the work should stop.

```

class MainViewModel : ViewModel() {

    fun fetch() {
        viewModelScope.launch {
            val user = fetchUser()
            // show user
        }
    }

    suspend fun fetchUser(): User {
        return withContext(Dispatchers.IO) {
            // fetch user
            // return user
        }
    }
}

```

Same idea, different owner.

Which one should you use?

Mostly `viewModelScope`. A ViewModel survives screen rotation, an Activity does not. So work started in `viewModelScope` is not thrown away when the user rotates the phone.

Use `lifecycleScope` for work that is truly tied to the UI itself.

These scopes come from the Android KTX extension libraries. Make sure the required dependencies are added to your project.

Why GlobalScope is dangerous

```
GlobalScope.launch {  
    // this runs until the whole app process dies  
}
```

`GlobalScope` is tied to the **application**, not to any screen.

- Nothing cancels it when the screen closes.
- It can hold references to a dead Activity → memory leak.
- In tests, it is very hard to control.

I used it in earlier parts only because it is short and keeps examples readable. In real code, do not.

Two scope helpers you should know

These are not "where to launch", they are "how to group work inside a coroutine".

`coroutineScope { }` - if any child fails, cancel all the others.

`supervisorScope { }` - if a child fails, the other children keep going.

Analogy: a group project.

- `coroutineScope` = *"if one member fails, the whole submission is cancelled."*
- `supervisorScope` = *"if one member fails, the rest still submit their parts."*

We will use both in Part 9.

Thumb-rules

- A scope decides **when the coroutine dies**.
- Cancel the scope → every coroutine inside it is cancelled.
- Activity → `lifecycleScope` . ViewModel → `viewModelScope` .
- Prefer `viewModelScope` - it survives rotation.
- Avoid `GlobalScope` in real projects.

Next: [CoroutineContext](#)

That's it for now.

Part 8: CoroutineContext

We have used Dispatchers. We have used Jobs. We have used scopes.

Now let's see the thing that holds them all together.

`CoroutineContext` is an interface in Kotlin Coroutines that defines the **context**, or the **environment**, in which a coroutine runs.

The backpack analogy

Every coroutine carries a small **backpack**.

Inside the backpack there are up to four things:

Item	What it does
Dispatcher	Which thread the work runs on
Job	The lifecycle handle - cancel it, check if it's done
CoroutineName	A name, useful while debugging
CoroutineExceptionHandler	What to do if an uncaught exception happens

That backpack is the `CoroutineContext`.

You have been packing this backpack all along, without noticing.

```
launch(Dispatchers.IO) { }
```

Here you packed exactly one item: the Dispatcher. Kotlin filled the rest with defaults.

Combining items with +

Each item (`Job` , `Dispatcher` , `CoroutineName` , `CoroutineExceptionHandler`) is a `CoroutineContext.Element`.

Elements are combined with the plus operator.

Just a Dispatcher:

```
launch(Dispatchers.IO) {  
    // work on IO thread  
}
```

Dispatcher + a name:

```
launch(Dispatchers.IO + CoroutineName("FetchUser")) {  
    // work on IO thread, named for easier debugging  
}
```

Dispatcher + name + exception handler:

```
val handler = CoroutineExceptionHandler { _, exception ->
    Log.d(TAG, "$exception handled!")
}

launch(Dispatchers.IO + CoroutineName("FetchUser") + handler) {
    // fully described environment
}
```

Pack it once, reuse it:

```
val myContext = Dispatchers.IO + CoroutineName("FetchUser") + handler

launch(myContext) {
    // same environment, reused
}
```

Cleaner when several coroutines share the same setup.

Why + and not a list?

Because + also means **replace**.

```
val context = Dispatchers.IO + Dispatchers.Default
// Dispatchers.Default wins
```

Only one item of each type can be in the backpack. Add a second Dispatcher and it **replaces** the first one.

Analogy: the backpack has one labelled pocket per item. Put a new water bottle in the bottle pocket and the old one comes out.

The context is inherited

Start a coroutine inside another coroutine, and the child gets a **copy of the parent's backpack**.

```
launch(Dispatchers.IO + CoroutineName("Parent")) {
    launch {
        // this child also runs on Dispatchers.IO
    }
}
```

The child did not ask for `Dispatchers.IO`. It inherited it.

Anything you pass explicitly to the child overrides just that one item.

```
launch(Dispatchers.IO + CoroutineName("Parent")) {  
    launch(Dispatchers.Main) {  
        // Dispatcher is Main now, but name still comes from the parent  
    }  
}
```

One exception: the **Job** is never inherited. Every coroutine gets its own new **Job**, which becomes a **child** of the parent's **Job**.

That parent-child chain of Jobs is what makes cancellation flow downward - cancel the parent and every child dies with it. That is the machinery behind Part 7.

Where you actually feel this

Most days you write `launch(Dispatchers.IO)` and never think about context.

You will care about it when you:

- Want readable names in a debug log → `CoroutineName`
- Want one place to catch errors → `CoroutineExceptionHandler`
- Want to cancel a specific group of work → a custom `Job`

Thumb-rules

- `CoroutineContext` = the backpack a coroutine carries.
- Four items: `Job`, `Dispatcher`, `CoroutineName`, `CoroutineExceptionHandler`.
- Combine with `+`. Same type added twice → the newer one replaces the older.
- Children inherit the parent's context, except the `Job`.

Next: [Exception Handling](#)

That's it for now.

Part 9: Exception Handling

Exception handling is another important topic. We must learn this.

Network calls fail. Servers go down. Wi-Fi drops. It **will** happen.

When using launch

Take a suspend function that might fail:

```
suspend fun fetchUserAndSaveInDatabase() {
    withContext(Dispatchers.IO) {
        // fetch user
        // save in database
    }
}
```

Way 1: try-catch

```
launch(Dispatchers.Main) {
    try {
        fetchUserAndSaveInDatabase() // work on IO thread, back on UI thread
    } catch (exception: Exception) {
        Log.d(TAG, "$exception handled!")
    }
}
```

Simple, and it reads exactly like normal Kotlin code. This is the one you will use most.

Way 2: CoroutineExceptionHandler

First, create the handler:

```
val handler = CoroutineExceptionHandler { _, exception ->
    Log.d(TAG, "$exception handled!")
}
```

Then attach it while launching:

```
launch(Dispatchers.Main + handler) {
    fetchUserAndSaveInDatabase()
}
```

Any exception inside will be caught by the handler.

Analogy: `try-catch` is a bucket you hold under one specific pipe. `CoroutineExceptionHandler` is the drain at the bottom of the room - it catches whatever leaks from anywhere.

Use the handler when several coroutines should report failures in the same way.

When using async

Remember from Part 5: an exception inside `async` is **silent** until you call `await()`.

So the `try-catch` goes **around** `await()`:

```
val deferredUser = async {
    fetchUser()
}

try {
    val user = deferredUser.await()
} catch (exception: Exception) {
    Log.d(TAG, "$exception handled!")
}
```

Wrapping only the `async { }` block does nothing. The envelope is opened at `await()`.

Real case: two network calls, one after another

```
launch {
    try {
        val users = getUsers()
        val moreUsers = getMoreUsers()
    } catch (exception: Exception) {
        Log.d(TAG, "$exception handled!")
    }
}
```

If either call fails, we jump straight to the catch block. We lose both results.

Sometimes that is fine. Sometimes it is not.

Real case: I want a partial result

Suppose we want an **empty list** for the call that failed, and still keep the result of the other one.

Give each call its own `try-catch`:

```

launch {
    val users = try {
        getUsers()
    } catch (e: Exception) {
        emptyList<User>()
    }

    val moreUsers = try {
        getMoreUsers()
    } catch (e: Exception) {
        emptyList<User>()
    }
}

```

Now one failure does not wipe out the other result.

Now in parallel - `coroutineScope`

Same two calls, but running at the same time:

```

launch {
    try {
        coroutineScope {
            val usersDeferred = async { getUsers() }
            val moreUsersDeferred = async { getMoreUsers() }
            val users = usersDeferred.await()
            val moreUsers = moreUsersDeferred.await()
        }
    } catch (exception: Exception) {
        Log.d(TAG, "$exception handled!")
    }
}

```

If any call fails, we go to the catch block - and the other call is **cancelled** too.

Partial result in parallel - `supervisorScope`

Want the other call to survive? Use `supervisorScope`, with a `try-catch` on each individual `await()`:


```

launch {
    supervisorScope {
        val usersDeferred = async { getUsers() }
        val moreUsersDeferred = async { getMoreUsers() }

        val users = try {
            usersDeferred.await()
        } catch (e: Exception) {
            emptyList<User>()
        }

        val moreUsers = try {
            moreUsersDeferred.await()
        } catch (e: Exception) {
            emptyList<User>()
        }
    }
}

```

One fails, the other still delivers.

coroutineScope vs supervisorScope

This is the heart of it:

`coroutineScope` cancels **everything** when any one of its children fails. `supervisorScope` lets the **other children continue** when one fails.

Analogy - a group project:

- `coroutineScope` = "If one member fails, the whole submission is cancelled." Strict team.
- `supervisorScope` = "If one member fails, everyone else still submits their part." Forgiving team.

Analogy - loading a screen:

- Loading the user profile fails → the screen is useless → `coroutineScope`.
- Loading "recommended for you" fails, but the main feed loaded fine → still show the feed → `supervisorScope`.

Conclusion

- While **NOT** using `async`, go with `try-catch` or `CoroutineExceptionHandler`.
- While using `async`, in addition to `try-catch`, you have two options: `coroutineScope` and `supervisorScope`.
- With `async`, use **`supervisorScope` + individual `try-catch`** when you want to continue with the other tasks if one fails.
- With `async`, use **`coroutineScope` + a top-level `try-catch`** when you do **not** want to continue if any of them fails.

Thumb-rules

- `launch` crashes loudly. `async` fails silently until `await()` .
- With `async` , always wrap `await()` , not the `async { }` block.
- All-or-nothing → `coroutineScope` .
- Best-effort, partial results → `supervisorScope` .

Next: [Flow - The Basics](#)

That's it for now.

Part 10: Flow - The Basics

Everything so far returned **one** value.

```
suspend fun fetchUser(): User
```

Call it, wait, get one user. Done.

But some things are not one value.

- A search box - the user types, and types, and types.
- A database table - rows change while the screen is open.
- A download - 10%, 40%, 70%, 100%.

For those, we need something that can give us **many values over time**.

That is **Flow**.

The analogy

- A **suspend function** is a **parcel**. It arrives once. You open it. Story over.
- A **Flow** is a **water pipe**. Open the tap and water keeps coming until you close it.

Or, if you prefer:

- suspend function = a **photo**
- Flow = a **video**

The smallest possible Flow

```
fun getNumbers(): Flow<Int> = flow {  
    emit(1)  
    emit(2)  
    emit(3)  
}
```

`emit()` = push one value into the pipe.

Now use it:

```
lifecycleScope.launch {  
    getNumbers().collect { value ->  
        Log.d(TAG, "Got $value")  
    }  
}
```

`collect { }` = stand at the end of the pipe and catch whatever comes out.

Output:

```
Got 1
Got 2
Got 3
```

The most important rule of Flow

A Flow does nothing until you collect it.

Writing `getNumbers()` runs zero code. The `flow { }` block only starts when someone calls `collect`.

This is called being **cold**.

Analogy: a pipe with the tap closed. The water is available, but nothing flows until you open the tap. `collect` is opening the tap.

Two collectors = two separate streams, each starting from the beginning. Two people, two taps, two glasses.

Flow is built on suspend

Because a Flow can pause between emissions, `collect` is a **suspend function**.

That means: **you can only collect a Flow inside a coroutine.**

Everything you learned in Parts 3 to 7 still applies here. Flow is not a new world - it is coroutines with more than one value.

Operators - changing the water on the way

You can transform values while they travel through the pipe.

```
getNumbers()
    .map { it * 10 }           // 10, 20, 30
    .filter { it > 10 }       // 20, 30
    .collect { value ->
        Log.d(TAG, "Got $value")
    }
```

Output:

```
Got 20
Got 30
```

Analogy: a filter attached to the pipe. Water goes in, cleaner water comes out. The pipe itself never changed.

Only `collect` actually starts the flow. `map` and `filter` just describe what should happen.

Choosing the thread

Same idea as Dispatchers, one new word.

```

getNumbers()
    .flowOn(Dispatchers.IO)    // the emitting side runs on IO
    .collect { value ->
        // collected on whichever thread the coroutine is on
    }

```

`flowOn` changes the thread for everything **above** it in the chain. The `collect` stays where your coroutine is.

StateFlow - a Flow that remembers

A plain Flow does not remember the last value. If you start collecting late, you missed everything.

`StateFlow` always holds the **current value**, and gives it to anyone who starts collecting.

```

class MainViewModel : ViewModel() {
    private val _uiState = MutableStateFlow<UiState>(UiState.Loading)
    val uiState: StateFlow<UiState> = _uiState

    fun load() {
        viewModelScope.launch {
            _uiState.value = UiState.Loading
            val user = fetchUser()
            _uiState.value = UiState.Success(user)
        }
    }
}

```

Analogy:

- **Flow** = a live radio broadcast. Tune in late, you missed the song.
- **StateFlow** = a scoreboard. Whenever you look at it, the current score is right there.

This is why `StateFlow` is the standard way to hold UI state in a ViewModel. Rotate the phone, the screen re-collects, and the current state is still there.

Quick comparison

	Gives	Cold or hot	Remembers last value
<code>suspend fun</code>	One value	-	-
<code>Flow</code>	Many values	Cold - starts on <code>collect</code>	No
<code>StateFlow</code>	Many values	Hot - always alive	Yes

Thumb-rules

- One value → suspend function. Many values over time → Flow.
- `emit()` puts values in. `collect { }` takes them out.
- A Flow is cold - nothing runs until you collect.
- `collect` is a suspend function, so it needs a coroutine.

- `flowOn` picks the thread for the emitting side.
- UI state in a ViewModel → `StateFlow`.

You made it

Ten parts. Look back at what you now understand:

1. Coroutines are functions cooperating.
2. They exist because callbacks and frozen UIs hurt.
3. `suspend` means pause and resume, not block.
4. Dispatchers pick the thread.
5. `launch` fires and forgets, `async` returns a result.
6. `withContext` shifts context without a new coroutine.
7. Scopes decide when coroutines die.
8. `CoroutineContext` is the backpack they carry.
9. Failures are handled with `try-catch`, handlers, and the right scope.
10. Flow handles many values over time.

Now go write some code. Re-reading is not learning - typing is.

That's it for now.